



Урок - Операционни системи

Клас: 12б | Предмет: Операционни системи УП (ИУЧ - СПП)

1. Виртуални машини и инсталиране на операционни системи

1.1. Какво е виртуална машина

Виртуална машина (VM) е софтуерна среда, която емулира (имитира) физически компютър.

- Позволява инсталиране на операционна система „вътре“ в друга ОС.
- Работи чрез хипервайзор (VirtualBox, VMware, Hyper-V и др.).
- Гост-ОС (guest) не знае, че не работи директно върху хардуера.

Виртуалната машина е **софтуерна среда, която емулира физически компютър** и позволява изпълнение на операционни системи в изолирана среда.

1.2. Защо инсталираме ОС на виртуална машина

- Можем да **тестваме различни операционни системи** без да променяме основната.
- По-безопасно е – експерименти, вируси, грешни настройки не чупят реалната ОС.
- Лесно връщане към предишно състояние чрез **snapshot**.

Основното предимство на виртуалните машини е, че **може да се тестват различни операционни системи без промяна на основната**, което прави експериментирането безопасно и обратимо.

2. Основни конзолни команди (Linux / Windows / PowerShell)

2.1. Команди за потребител и помощ

Действие	Команда	Описание
Кой е текущият потребител	<code>whoami</code>	Показва името на текущо влезлия потребител
Помощ / документация за команди	<code>help</code> (Windows), <code>man</code> команда (Linux)	Показва помощна информация и документация за команди

2.2. Работа с файлове и директории

Действие	Windows	Linux / Unix	Пример
Списък с файлове/ директории	<code>dir</code>	<code>ls</code>	<code>ls -la</code> (подробен списък)
Навигиране между директории	<code>cd</code>	<code>cd</code>	<code>cd Documents</code>
Създаване на директория	<code>mkdir</code>	<code>mkdir</code>	<code>mkdir projekti</code>
Копиране на файл	<code>copy source dest</code>	<code>cp source dest</code>	<code>cp file.txt backup.txt</code>

Преместване/ преименуване	<code>move source dest</code>	<code>mv source dest</code>	<code>mv old.txt new.txt</code>
Изтриване на файл	<code>del file</code>	<code>rm file</code>	<code>rm file.txt</code>
Показване съдържание на текстов файл	PowerShell: <code>Get- Content file.txt</code>	<code>cat file.txt</code>	<code>cat config.txt</code>
Символна/ твърда връзка (link)	<code>mklink (cmd), New-Item - ItemType SymbolicLink (PowerShell)</code>	<code>ln , ln - s</code>	<code>ln -s /path/to/file link</code>

3. Наблюдение на процеси и използване на паметта

3.1. Инструменти за процеси

- **Linux / Unix:**
 - `ps` – списък на процесите (моментна снимка).
 - `top` / `htop` – динамичен списък с процеси, използване на CPU и RAM.
- **Windows:**
 - **Task Manager (Диспечер на задачите)** – процеси, CPU, памет, диск, мрежа.

Тези инструменти са основни за мониторинг и управление на системните ресурси и процеси.

3.2. Памет и системна статистика

- `free` – показва обща, заета и свободна памет.
- `vmstat` – статистика за памет, процеси, I/O, swap и др.
- `/proc/meminfo` – файл с подробна информация за паметта; преглежда се с `cat /proc/meminfo`.

Резюме на инструментите за мониторинг:

- **ps / top:** показват списък с процеси, PID, използване на CPU и RAM
- **Task Manager:** графично показва процеси и ресурси в Windows
- **free / vmstat / /proc/meminfo:** дават подробна информация за използването на паметта

4. Потребители, групи и права за достъп

4.1. Обикновен потребител vs. администратор

- **Обикновен потребител** – има ограничени права; работи основно в своята домашна директория.
- **Администратор** – root (Linux) или Administrator (Windows); пълен контрол над системата.
- Администраторска сметка се използва за:
 - инсталиране/премахване на софтуер;
 - създаване/изтриване на потребители и групи;
 - промяна на системни настройки.

Разбирането на разликата между обикновен потребител и администратор е критично за безопасността и правилното управление на системата.

4.2. Права за достъп до файлове и директории

- Права: **read (r)**, **write (w)**, **execute (x)**.
- Контролът на достъпа определя кой може да чете, променя или изпълнява даден файл.
- Неправилни права могат да доведат до:
 - изтичане на чувствителна информация;

- изтриване/повреда на важни файлове от обикновен потребител;
- невъзможност дадена програма да се стартира.

Защо правилните права са важни: Те защитават системата от неоторизиран достъп, предотвратяват случайно изтриване на важни файлове и осигуряват, че само оторизирани потребители могат да изпълняват критични операции.

4.3. Команди за управление на права: `chmod` и `chown`

chmod – променя правата за достъп до файлове и директории в Linux.

- Правата се задават за три групи: **собственик (owner)**, **група (group)** и **други (others)**.
- Всяка група може да има права за четене (r), писане (w) и изпълнение (x).
- Правата могат да се задават с числа (октална система) или символи.

Примери за `chmod`:

```
chmod 755 file.txt      # собственик: rwx, група: r-x, др  
chmod 644 file.txt      # собственик: rw-, група: r--, дру  
chmod +x script.sh      # добавя право за изпълнение за вс  
chmod u+w file.txt      # добавя право за писане за собсте
```

Октални числа: 4=read, 2=write, 1=execute. Сумата дава правата (напр. 7=4+2+1=rwx).

chown – променя собственика и/или групата на файл.

Примери за `chown`:

```
chown user:group file.txt  # променя собственика и гр  
chown user file.txt        # променя само собственика  
chown :group file.txt      # променя само групата
```

Командите `chmod` и `chown` са основни за управлението на правата за достъп в Linux системи. Разбирането на различните права (`read`, `write`, `execute`) и как да ги задаваме е важно за правилното управление на файловата система.

5. Shell скриптове и автоматизация

5.1. Какво е shell скрипт

- Файл с поредица от команди, които се изпълняват автоматично от shell-a.
- Пример: `bash` скрипт в Linux или `PowerShell` скрипт в Windows.

Пример (`bash`):

```
#!/bin/bash
echo "Hello"
pwd
ls
```

Основни стъпки за създаване и изпълнение на `shell` скрипт:

1. Създавате файл, напр. `script.sh`.
2. Добавяте команди вътре.
3. Правите файла изпълним: `chmod +x script.sh`.
4. Стартирате: `./script.sh`.

6. Пакетни мениджъри и инсталиране на софтуер

6.1. Основна идея

Пакетен мениджър е инструмент, който автоматично изтегля, инсталира, обновява и премахва софтуер.

ОС	Пакетен мениджър	Пример
Debian / Ubuntu	apt	<code>sudo apt install firefox</code>
Fedora / Red Hat	yum / dnf	<code>sudo yum install httpd</code>
Windows 10/11	winget	<code>winget install Notepad++</code>

Структура на команда за инсталиране:

- `sudo apt install firefox` – `sudo` изпълнява с администраторски права, `apt` е пакетният мениджър, `install` е действието, `firefox` е името на програмата
- `winget install Notepad++` – `winget` е пакетният мениджър за Windows, `install` е действието, `Notepad++` е името на програмата

Пакетният мениджър автоматично изтегля нужните файлове и зависимости от хранилището, което прави инсталирането на софтуер много по-лесно от ръчното изтегляне и инсталиране.

7. Практически примери и упражнения

7.1. Наблюдение на процеси и памет

Какво е процес? Процесът е изпълняваща се програма в операционната система. Всеки процес има уникален идентификатор (PID) и използва системни ресурси като CPU и памет.

Защо е важно да наблюдаваме процесите? Наблюдението на процесите ни позволява да:

- Идентифицираме програми, които използват твърде много ресурси
- Затворим зависнали или проблемни процеси
- Оптимизираме производителността на системата
- Откриваме потенциални проблеми със сигурността

Инструменти за наблюдение:

- `ps` и `top` показват списък с процеси, техните PID, потребител, използване на CPU и RAM
- Task Manager в Windows дава графично представяне на процеси, натоварване на процесора, памет и други ресурси
- `free`, `vmstat` и `/proc/meminfo` предоставят подробна информация за използването на паметта – обща/свободна памет, swap, кеш и др.

7.2. Работа с файлове и директории

Операционните системи позволяват работа с файлове и директории чрез конзолни команди. Това е по-ефективно и мощен начин за управление на файловата система в сравнение с графичния интерфейс.

Основни команди:

- `cd` – променя текущата директория (напр. `cd Documents`)
- `ls` (Linux) / `dir` (Windows) – показва списък с файлове и директории
- `mkdir` – създава нова директория (напр. `mkdir projekti`)
- `cp` (Linux) / `copy` (Windows) – копира файлове (напр. `cp file.txt backup.txt`)
- `mv` (Linux) / `move` (Windows) – премества или преименува файлове
- `rm` (Linux) / `del` (Windows) – изтрива файлове

Тези команди са еднакво важни в Linux и Windows (cmd/PowerShell) и формират основата на работата с файловата система чрез командния ред.

7.3. Потребители, групи и права за достъп

Обикновен потребител vs. администратор:

- **Обикновен потребител** има ограничени права и работи основно в своята домашна директория
- **Администратор** (root в Linux или Administrator в Windows) има пълен контрол над системата

Защо създаваме групи? Групите улесняват управлението на правата за достъп. Вместо да задаваме права за всеки потребител поотделно, можем да създадем група и да зададем права за цялата група наведнъж.

Права за достъп: Правата (read/write/execute) се свързват с потребители и групи. Например, в обща папка на фирма/клас всички потребители могат да имат достъп за четене, но само администраторът може да изтрива или редактира файлове.

Рискове от неправилни права: Неправилно зададени права могат да доведат до изтичане на чувствителна информация, изтриване на важни файлове от неоторизирани потребители, или до невъзможност важна програма да се стартира.

7.4. Управление на права с `chmod` и `chown`

Командата `chmod` променя правата за достъп до файлове и директории в Linux. Правата се задават за три групи:

- **Собственик (owner)** – потребителят, който притежава файла
- **Група (group)** – потребителите, които са част от групата на файла
- **Други (others)** – всички останали потребители

Видове права:

- **read (r)** – право за четене на файл или списък с файлове в директория
- **write (w)** – право за промяна/изтриване на файл или създаване на файлове в директория
- **execute (x)** – право за изпълнение на файл (скрипт/програма) или влизане в директория

Примери за `chmod`:

- `chmod 755 script.sh` – собственикът има всички права (rwx), групата и другите имат четене и изпълнение (r-x)
- `chmod 644 file.txt` – собственикът може да чете и пише (rw-), останалите могат само да четат (r--)

Командата `chown` променя собственика и/или групата на файл:

- `chown user:group file.txt` – променя и собственика, и групата

- `chown user file.txt` – променя само собственика

7.5. Създаване и изпълнение на shell скрипт

Стъпки за създаване на shell скрипт:

1. Създавате файл с разширение `.sh` (напр. `script.sh`)
2. Добавяте команди вътре (първият ред обикновено е `#!/bin/bash` за bash скриптове)
3. Задавате права за изпълнение: `chmod +x script.sh`
4. Стартирате скрипта: `./script.sh`

Защо скриптовете са полезни? Те автоматизират повтарящи се задачи, което спестява време и намалява вероятността от грешки. Скриптовете могат да се използват за резервни копия, системна поддръжка, обработка на данни и много други задачи.

7.6. Роли на потребители и инсталиране на софтуер

Обикновен потребител: Има ограничени права и работи основно в своята домашна директория. Не може да инсталира системен софтуер или да променя критични системни настройки.

Администратор: Има пълен контрол над системата. Може да:

- Инсталира и деинсталира програми
- Променя системни настройки (час, мрежа, драйвери)
- Създава и изтрива потребители и групи
- Променя правата за достъп до системни файлове

Защо не е добре да работим постоянно като администратор? Работата като администратор увеличава риска от случайни грешки, които могат да повредят системата, и прави компютъра по-уязвим към вируси и злонамерен софтуер.

Инсталиране на софтуер с пакетен мениджър:

Примерна команда: `sudo apt install firefox`

- `sudo` – изпълнява командата с администраторски права (Linux)
- `apt` – името на пакетния мениджър (за Debian/Ubuntu)
- `install` – действието (инсталиране)
- `firefox` – името на програмата за инсталиране

Пакетният мениджър автоматично изтегля нужните файлове и зависимости от хранилището, което прави процесът много по-лесен от ръчното инсталиране.

Резюме на урока

В този урок научихме за:

- Виртуални машини и тяхното използване
- Основни конзолни команди за работа с файлове и директории
- Наблюдение на процеси и използване на паметта
- Управление на потребители, групи и права за достъп
- Създаване и изпълнение на shell скриптове
- Използване на пакетни мениджъри за инсталиране на софтуер